MalariaScope

Low-Level Design

Kiran Kumar P
DA24B008

This is the implementation spec. I checked Section 3 against the backend's live OpenAPI schema (`/docs`): every endpoint and response field matches.

1 Source tree

```
src/malariascope/
  common.py          params, logging, Timer, write_json
  image.py           to_model_input(): resize shared by pipeline and backend
  labels.py          CLASS_NAMES, POSITIVE: shared by pipeline and backend
  model.py           build_model()
  tracking.py        MLflow URI, run tags (git commit, DVC data hash)
  data/ingest.py     TFDS -> PNG + manifest
  data/validate.py   Ray Data image checks + dedup
  data/split.py      stratified seeded split; adds feedback to train only
  data/feedback.py   exports the feedback store into the DVC snapshot
  data/preprocess.py Ray Data resize -> .npz
  eda.py             dataset figures
  train.py           CNN training, MLflow logging
  evaluate.py        test metrics, val-tuned threshold
  promote.py         registry: challenger vs champion gate
  rollback.py        champion <- previous_champion
backend/             FastAPI service (section 3)
frontend/            Streamlit UI (section 5)
airflow/dags/        training and feedback-retrain DAGs (section 6)
monitoring/          Prometheus, Alertmanager and Grafana configuration (section 7)
scripts/             traffic simulator, serving smoke test, MLflow store snapshot
```

2 Pipeline stages (`dvc.yaml`)

Every stage runs as `PYTHONPATH=src python -m <module>`.

Stage	Module	Inputs	Outputs
ingest	`malariascope.data.ingest`	TFDS `malaria`, `params.data.dataset`	`data/raw/`, `results/ingest.json`
validate	`malariascope.data.validate`	`data/raw`, `params.validate`	`data/validated/`, `results/validate.json`
split	`malariascope.data.split`	`data/validated`, `params.data.seed/split`	`data/split/`, `results/split.json`
preprocess	`malariascope.data.preprocess`	`data/raw`, `data/split`, `params.preprocess`	`data/processed/{train,val,test}.npz`, `results/preprocess.json`
eda	`malariascope.eda`	`data/raw`, `data/validated`	`docs/figures/*.png`
train	`malariascope.train`	`data/processed`, `params.train`	`models/model.keras`, `results/train.json`, MLflow run
evaluate	`malariascope.evaluate`	model, `data/processed`, `params.evaluate`	`results/evaluate.json`, `results/plots/*.png`
promote	`malariascope.promote`	`results/evaluate.json`, `params.promote`	registry version + aliases, `results/promote.json`

3 Backend REST API (FastAPI, port 8000)

Run with `PYTHONPATH=src:backend uvicorn app.main:app --port 8000`. Every response carries the `X-Request-ID` and `X-Process-Time-ms` headers. All JSON responses are UTF-8, and every error has the same shape:

```
{"error": "<machine_code>", "detail": "<human readable message>", "request_id": "<uuid4>"}
```

3.1 POST /predict

Classifies one cell image.

Request	`multipart/form-data`, field `file`: PNG or JPEG, ≤ 5 MB
200	`PredictResponse`
400 `invalid_image`	file isn't a decodable PNG/JPEG
413 `file_too_large`	file > 5 MB
422 `validation_error`	`file` field missing
503 `model_unavailable`	MLflow model server unreachable or erroring

```
PredictResponse {
  "request_id": "string (uuid4)",
  "label": "parasitized | uninfected",
  "is_parasitized": "bool",
  "p_parasitized": "float 0-1",
  "confidence": "float 0-1, probability of the returned label",
  "threshold": "float, decision threshold of the serving model version",
  "model_version": "string",
  "latency_ms": "float"
}
```

`confidence` can be below 0.5 when the tuned threshold is below 0.5. The UI flags those results for a manual re-check.

Steps:

1. Check the content type and size.
2. Decode with PIL and run `to_model_input(img, image_size)`.
3. POST `{"instances": [[...uint8 64x64x3...]]}` to `MODEL_SERVER_URL/invocations`.
4. Apply the threshold: the label is `parasitized` if `p_parasitized >= threshold`.
5. Update the Prometheus metrics (Section 7.1).

3.2 POST /feedback

Stores a user's correction or confirmation for retraining.

Request	`multipart/form-data`: `file` (the same image), `request_id` (str), `predicted_label`, `correct_label` (both `parasitized`/`uninfected`), `p_parasitized` (float), `model_version` (str)
201	`{"stored": true, "feedback_id": int, "agreed": bool, "feedback_count": int}`
400 `invalid_image` / `invalid_label`	
413 `file_too_large`	
500 `storage_error`	writing the image or the DB row failed

3.3 GET /feedback/stats

Returns 200 `{"total": int, "agreed": int, "disagreed": int, "since_last_export": int}`. Airflow uses it to decide on retraining.

3.4 GET /model/info

Returns 200 with the body below, or 503 `model_unavailable`.

```
{"name": "malariascope", "alias": "champion", "version": "string",
 "threshold": float, "image_size": int, "test_accuracy": float,
 "test_recall_parasitized": float, "git_commit": "string", "data_md5": "string",
 "class_names": ["parasitized", "uninfected"]}
```

The values come from the MLflow registry (the tags on the version behind `@champion`; `image_size` comes from the model signature) and are cached for `MODEL_INFO_TTL` seconds. If the registry is down, the last known info is served.

3.5 GET /health and GET /ready

- `/health`: always 200 `{"status": "ok"}` while the process is up (liveness).
- `/ready`: 200 `{"status": "ready", "model_server": "up"}`, or 503 `{"status": "not_ready", "model_server": "down"}`. The UI uses it for the service status. The Docker healthcheck uses `/health`, so a model server outage shows in the UI without the backend being restarted.

3.6 GET /metrics

Prometheus text format (Section 7.1).

3.7 Configuration

The defaults suit running outside Docker. `docker-compose.yml` sets the service addresses and `FEEDBACK_DIR=/data/feedback`.

Variable	Default	Purpose
`MODEL_SERVER_URL`	`http://localhost:5001`	MLflow model server
`MLFLOW_TRACKING_URI`	`http://localhost:5000`	registry lookup for `/model/info`
`FEEDBACK_DIR`	`feedback`	feedback images and DB
`MAX_UPLOAD_MB`	`5`	upload limit
`MODEL_INFO_TTL`	`60`	seconds to cache registry info
`MODEL_NAME`, `MODEL_ALIAS`	`malariascope`, `champion`	which registered model to serve
`MODEL_REQUEST_TIMEOUT`	`10`	seconds to wait for the model server
`LOG_LEVEL`	`INFO`	logging level

4 Feedback store

```
$FEEDBACK_DIR/
  feedback.db                          SQLite metadata
  images/<correct_label>/<id>.png      image files (easy to dvc add and to diff)
```

```
CREATE TABLE feedback (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  ts              REAL    NOT NULL,      -- unix time
  request_id      TEXT    NOT NULL,
  predicted_label TEXT    NOT NULL,
  correct_label   TEXT    NOT NULL,
  p_parasitized   REAL    NOT NULL,
  model_version   TEXT    NOT NULL,
  image_path      TEXT    NOT NULL,      -- relative to FEEDBACK_DIR
  exported        INTEGER NOT NULL DEFAULT 0   -- 1 once copied into a training snapshot
);
```

5 Frontend (Streamlit, port 8501)

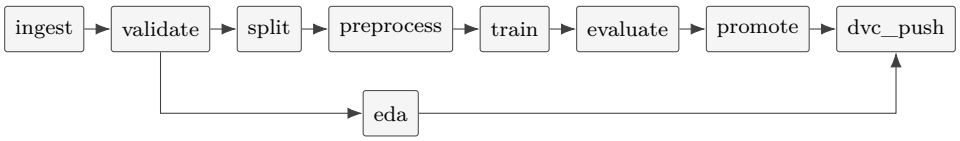
- It reads `BACKEND_URL` from the environment and has no other link to the backend.

- **Predict page:** upload or pick a sample → preview → Analyse → label, confidence bar and threshold → thumbs-up / thumbs-down feedback, sent once per result. Below 70% confidence the result is flagged for a manual re-check.
- **Pipeline page:** Airflow DAG runs with *Run now* buttons, per-stage state and duration, Ray throughput, the MLflow champion, versions and recent runs, feedback stats, and links to the Airflow, MLflow, Grafana, Prometheus and Alertmanager UIs.
- Error states: backend unreachable, model not ready, invalid file, file too large.

6 Airflow DAGs

Airflow 2.10.5 (`apache/airflow:2.10.5-python3.10`) runs as three compose services: `airflow-init` (DB migration and admin user), `airflow-webserver` (UI and REST API on :8080) and `airflow-scheduler`. The executor is `SequentialExecutor` on SQLite. Tasks run inside the scheduler container, on the repo bind-mounted at `/opt/project`. They use a separate venv (`/opt/pipeline`, from `airflow/requirements-pipeline.txt`), so the pipeline's TensorFlow, Ray, DVC and MLflow can't clash with Airflow's own pinned packages.

6.1 `training_pipeline` (manual, REST API, or started by `feedback_retrain`)



- **Tasks:** each task runs `dvc repro <stage>`, then prints the metrics JSON. The printed line becomes the task's XCom, which the Pipeline page reads (for example Ray throughput).
- **Skipping:** DVC skips stages whose inputs are unchanged, so a run with no new data takes under a minute.
- **One run at a time:** `max_active_runs=1` keeps DVC runs on the shared repo from overlapping. `retries=0` keeps failures visible.
- **Promotion:** `promote` only moves the `@champion` alias. The model server watches the alias (`mlflow-model-server/serve.py`, polling every 15 s) and restarts itself on the new version, so Airflow needs no access to other containers. Rollback works the same way.

6.2 `feedback_retrain` (every 30 minutes)



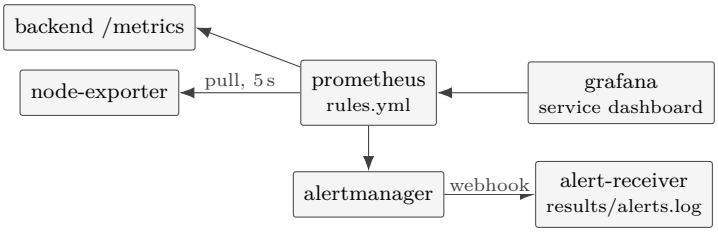
- `enough_feedback` (ShortCircuitOperator): `GET {BACKEND_URL}/feedback/stats`. It continues only if `since_last_export` $\geq$ `retrain.min_feedback` (`params.yaml`, 50). A trigger config `{"min_feedback": N}` overrides the threshold for one run. Otherwise the downstream tasks show as skipped.
- `export_feedback`: `python -m malariascope.data.feedback`, then `dvc add data/feedback`. It copies unexported items into `data/feedback/images/<label>/fb_<id>.png` and appends them to the cumulative `data/feedback/manifest.csv` (with the pixel SHA-1). It sets `exported=1` in `feedback.db`, and only after the snapshot is written.
- `start_training`: TriggerDagRunOperator. It doesn't wait for the training run, because under the SequentialExecutor a waiting task would hold the only slot.

6.3 How feedback enters training (split stage)

- **Train only:** feedback rows join the train split only (`source=feedback`). Val and test never change, so challenger and champion are compared on the same images.
- **Leakage guard:** feedback whose decoded pixels match a val or test image is dropped. The UI's sample images come from the test set.

- **Duplicates:** repeated feedback on the same cell keeps the latest label.
- **Order:** feedback rows are appended after the raw rows. With an empty snapshot, `train.npz` is byte-identical to before (checked on 3 Nov), and training is skipped.

7 Monitoring (Prometheus, Alertmanager, Grafana)



Service	Port	Image	Role
prometheus	9090	`prom/prometheus:v3.15.0`	scrapes every 5 s, evaluates `rules.yml` every 5 s
alertmanager	9093	`prom/alertmanager:v0.34.1`	groups alerts, applies inhibit rules, posts to the webhook
alert-receiver	internal 9095	`python:3.10-slim`	appends each notification to `results/alerts.log`
node-exporter	internal 9100	`prom/node-exporter:v1.12.1`	host CPU, memory, disk
grafana	3000	`grafana/grafana:12.4.14`	provisioned datasource and dashboards

7.1 Backend metrics (`GET /metrics`)

Metric	Type	Labels	Meaning
`malariascope_http_requests_total`	counter	method, path, status	requests by route template (not raw URL)
`malariascope_http_request_seconds`	histogram	method, path	end-to-end request latency
`malariascope_requests_in_flight`	gauge		requests being handled
`malariascope_predictions_total`	counter	label	class distribution of predictions
`malariascope_prediction_confidence`	histogram	label	probability of the returned label
`malariascope_inference_seconds`	histogram		model server round trip
`malariascope_input_mean_intensity`	histogram		mean pixel (0–255) of the resized input; drift signal
`malariascope_upload_bytes`	summary		upload size
`malariascope_feedback_total`	counter	agreed, correct_label	feedback; live accuracy = agreed share
`malariascope_errors_total`	counter	error	handled errors by error code (Section 3)
`malariascope_model_version`	gauge	alias	registry version behind `@champion`
`malariascope_model_threshold`	gauge		decision threshold in use
`malariascope_model_server_up`	gauge		1 if the model server answers `/ping`
`malariascope_feedback_pending`	gauge		feedback not yet exported for retraining

The last four are refreshed on every scrape, so they stay current without traffic, and `/metrics` still answers while the model server is down.

7.2 Drift baseline

The preprocess stage writes `train_intensity` {mean, std, p05, p95} of the per-image mean intensity of the train split to `results/preprocess.json`. On the current data it is mean 119.5, std 11.9. These two numbers are written into the `InputDrift` rule and the Grafana drift panel; if the training data changes a lot, update them there.

7.3 Alert rules (`monitoring/prometheus/rules.yml`)

Alert	Condition	For	Severity
BackendDown	`up{job="backend"} == 0`	30 s	critical
ModelServerDown	`malariascope_model_server_up == 0`	30 s	critical
PredictLatencyHigh	p95 `/predict` over 5 min > 0.5 s (normal: about 50 ms)	2 min	warning
HighErrorRate	5xx share over 2 min $> 5\%$	1 min	warning
InputDrift	$\lvert$15-min mean upload intensity $-$ training mean$\rvert >$ training std, with ≥ 20 uploads	1 min	warning
FeedbackAccuracyLow	agreed share of last hour's feedback < 0.8, with ≥ 10 items	1 min	warning
NodeExporterDown	`up{job="node"} == 0`	30 s	critical
HostCPUHigh / HostMemoryHigh	$> 90\%$	2 min	warning

- **Why the drift threshold:** the mean of 20 random test images varies by only about 2.7, so a shift of a whole training std (11.9) is very unlikely to be chance.
- **Inhibition (`alertmanager.yml`):** while `ModelServerDown` or `BackendDown` fires, the serving latency and error alerts are held back.

7.4 Grafana dashboard (`monitoring/grafana/dashboards/`, provisioned)

MalariaScope - Service (`/d/malariascope-service`, refresh 10 s):

- status row: champion version, model server up, requests/s, 5xx ratio, p95 latency, feedback waiting for retrain
- traffic and latency: requests by route and status, p50/p95 `/predict`, model server round trip
- predictions and feedback: predictions per minute by label, confidence distribution, live accuracy implied by feedback
- data drift: 15-min mean upload intensity against the training mean and the ± 1 std alert band, plus the upload intensity distribution
- firing alerts table (`ALERTS`) and host CPU and memory

Pipeline runs, task states, durations and logs are monitored in the Airflow UI and on the Streamlit Pipeline page (Section 5), not in Grafana.

8 Docker images

All Python images are multi-stage: a `python:3.10` builder and a `python:3.10-slim` runtime. The Airflow image is the exception, it starts from `apache/airflow:2.10.5-python3.10`.

Image	Entrypoint	User
`malariascope/backend:1.0`	`uvicorn app.main:app --host 0.0.0.0 --port 8000`	`appuser` (compose: host user)
`malariascope/frontend:1.0`	`streamlit run app.py --server.port=8501 --server.address=0.0.0.0`	`appuser`
`malariascope/mlflow:3.17.2`	`mlflow server --backend-store-uri sqlite:///mlflow.db --artifacts-destination ./mlartifacts`	compose: host user
`malariascope/model-server:1.0`	`python serve.py` (resolves `@champion`, runs `mlflow models serve`)	`appuser`
`malariascope/airflow:2.10.5`	`airflow webserver`, `airflow scheduler`, one-shot `airflow-init`	compose: host user

The upstream images are pinned and run with their default entrypoint and user: `prom/prometheus:v3.15.0`, `prom/alertmanager:v0.34.1`, `prom/node-exporter:v1.12.1` and `grafana/grafana:12.4.14`. The alert receiver runs `monitoring/alert-receiver/alert_receiver.py` on plain `python:3.10-slim`.

9 Environment variables

The backend variables are in Section 3.7. The others:

Variable	Default	Used by	Meaning
BACKEND_URL	http://localhost:8000	Streamlit	API base for all UI requests
MLFLOW_URL	http://localhost:5000	Streamlit	MLflow REST API for the Pipeline page
AIRFLOW_API_URL	http://localhost:8080	Streamlit	Airflow REST API
AIRFLOW_USER, AIRFLOW_PASSWORD	admin, admin	Streamlit	Airflow REST basic auth
BACKEND_TIMEOUT	30	Streamlit	seconds to wait for the backend
AIRFLOW_URL, MLFLOW_PUBLIC_URL, GRAFANA_URL, PROMETHEUS_URL, ALERTMANAGER_URL	http://localhost:<port>	Streamlit	links opened by the user's browser
MLFLOW_TRACKING_URI	http://localhost:5000	pipeline stages, model server	MLflow server
MODEL_NAME, MODEL_ALIAS	malariascope, champion	model server	model to follow
POLL_SECONDS	15	model server	how often to check the alias
PROJECT_DIR, PIPELINE_BIN	/opt/project, /opt/pipeline/bin	Airflow DAGs	mounted repo and pipeline venv
HOST_UID, HOST_GID	1000	Compose	user that owns bind-mounted files